

Iterative algorithms

The topic for this session is iteration, i.e. expressing algorithms using loops. We'll consider three tools to reason about such algorithms:

Loop invariants

for demonstrating partial correctness

Loop control variables

for keeping track of when to exit the loop

Order-of-growth ("big O ") notation

for measuring complexity (mainly worst-case time) - more next week

Correctness

The first step is to distinguish two kinds of correctness:

definition

Total correctness: for all inputs, the algorithm produces the desired result.

This is what we want, but it can be hard to establish in one go, so we aim for something easier:

definition

Partial correctness: if the algorithm terminates, it produces the desired result.

As an extreme example, an algorithm that loops forever is a partially correct solution for any problem.

However, separating the two simplifies the problem, because partial correctness is easier to prove, and the time analysis we're planning will imply termination:

- total correctness = partial correctness + termination
- worst-case time analysis $\Rightarrow$ termination

What stays the same: loop invariants

One of last week's exercises was to write an algorithm to find the value of the largest element of the array.

Step through the Largest algorithm:

n = 5

0

1

2

3

4

a

27

14

77

85

34

max

i

Reset

Single step

Big step

Run

Stop

Let's trace this algorithm on the sample array. The state consists of the array (which doesn't change here) and the variables **max** and **i**. We tabulate their values at each iteration:

	max	i
On entering the loop	27	1
After the first iteration	27	2
After the second iteration	77	3
After the third iteration	85	4
On exiting the loop	85	5

In order to investigate this algorithm, it is useful to identify a key property of the loop:

definition
A **loop invariant** is a property that holds before and after each execution of the loop (i.e. each time the execution reaches the WHILE line).

and the final value of **max**, namely 85, is returned.

In this case, a property that holds for each row of the table at right. We add it to our description of the algorithm:

Algorithm Largest(a[0..n-1])

max ← a[0]

i ← 1

invariant: $1 \leq i \leq n$ and max is largest of a[0] ...

a[i-1]

WHILE i < n

IF a[i] > max

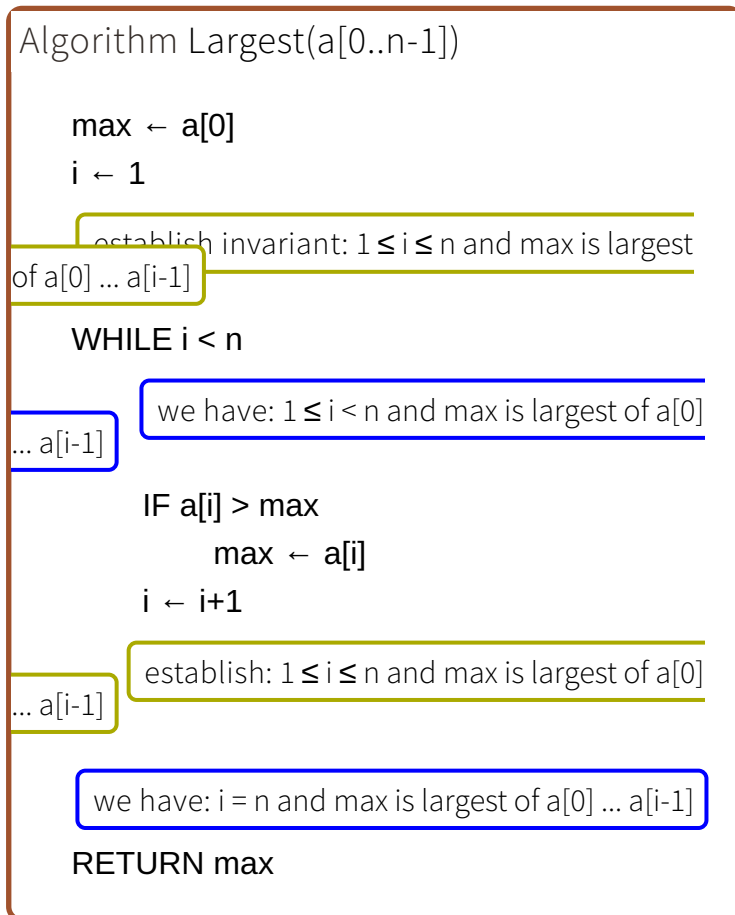
max ← a[i]

i ← i+1

RETURN max

The loop invariant is not executed; it is a comment.

In more detail, stating a loop invariant means we have to establish that property before the loop starts and at the end of the loop body, but it also gives us properties that hold at the start of the loop body and after the loop exits:

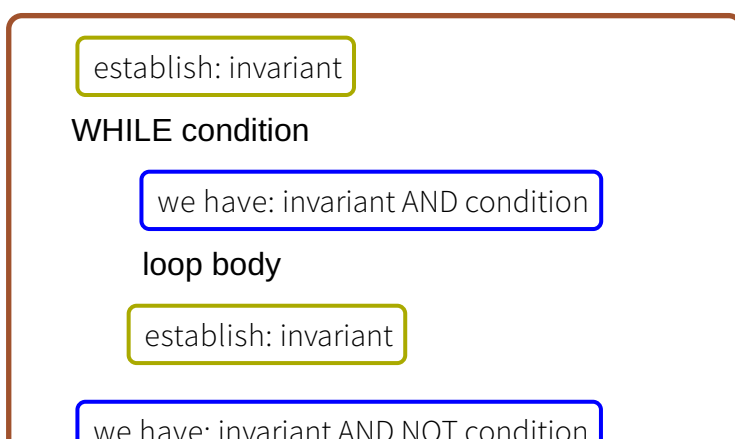


Robert Floyd introduced assertions and invariants in 1967.

You may have noticed that the idea of loop invariants is a form of mathematical induction.

and thus, in the case of the Largest algorithm, we know that after the loop, max is the largest of $a[0] \dots a[n-1]$.

In general:



This works even if the loop doesn't terminate, because we're only claiming partial correctness here. (But we'll establish termination later as a consequence of the complexity analysis.)

The invariant has to be true at the start and end of the body of the loop. It don't have to be true in the middle of the body (and often isn't, e.g. in the case of Largest).

Example: sequential search

Another of last week's exercises was an algorithm that takes an array and a value **x** and returns the index of the first occurrence of the value in the array, or **n** if the value is not present. Here's a simple version:

Algorithm SequentialSearch($a[0..n-1]$, x)

$i \leftarrow 0$

invariant: $0 \leq i \leq n$ and x is not in $a[0] \dots a[i-1]$

WHILE $i < n$ AND $a[i] \neq x$

$i \leftarrow i+1$

RETURN i

On exit from the loop, one of two things must be true:

- $i = n$ and x is not in the array, or
- $0 \leq i < n$ and $a[i] = x$ and x is not in $a[0] \dots a[i-1]$

so the algorithm is partially correct (and indeed totally correct).

Try stepping through the above algorithm with the search key 77 :

SequentialSearch($a[0..n-1]$, x)					
$n = 5$	0	1	2	3	4
a	27	14	77	85	34
x	77				
i					

$i \leftarrow 0$
WHILE $i < n$ AND $a[i] \neq x$
 $i \leftarrow i+1$
RETURN i

Reset Single step Big step Run Stop

Try using a different search key.

Nested loops

Consider the problem of testing whether all the elements of an array are different. We'll need to go through all the elements, For each element, we'll need to go through all the others and check that they're different from this one. That will require two loops, one inside the other. We place an invariant on each loop.

Algorithm AllDifferent($a[0..n-1]$)

$i \leftarrow 1$

$\text{diff} \leftarrow \text{true}$

invariant: $0 \leq i \leq n$ and $\text{diff} \equiv a[0] \dots a[i-1]$ all different

WHILE $i < n$

$j \leftarrow 0$

invariant: $0 \leq i \leq n$ and $\text{diff} \equiv a[0] \dots a[i-1]$ all different AND $a[i]$ different from $a[0] \dots a[j-1]$

WHILE $j < i$

IF $a[j] = a[i]$

$\text{diff} \leftarrow \text{false}$

$j \leftarrow j+1$

$i \leftarrow i+1$

RETURN diff

Identifying the invariant on each loop is a good way of documenting the logic of an iterative algorithm. Working the other way round can also be a good way to design algorithms: identify the invariants first, and then write code that preserves them. We'll see some examples of this in future weeks.

Step through the AllDifferent algorithm:

n = 4 0 1 2 3	AllDifferent(a[0..n-1])			
a	27	14	77	27
i				
diff				
j				

```

i ← 1
diff ← true
WHILE i < n
  j ← 0
  WHILE j < i
    IF a[j] = a[i]
      diff ← false
    j ← j+1
  i ← i+1
RETURN diff

```

What changes: loop control variables

Consider the algorithm Largest again:

Algorithm Largest(a[0..n-1])

```

max ← a[0]
i ← 1
invariant: 1 ≤ i ≤ n and max is largest of a[0] ...
a[i-1]
WHILE i < n
  IF a[i] > max
    max ← a[i]
  i ← i+1
RETURN max

```

The variable **i** here has a special role: it keeps track of our progress towards exiting the loop.

We can usually specify an upper bound on the number of remaining iterations as a function of the value of the loop control variable. In the case of Largest, the number of iterations remaining is $n-i$.

A **loop control variable** is variable that is

- initialized before the loop starts
- used in the test whether to continue looping
- modified in the body of the loop

Recall the AllDifferent algorithm, with one loop nested inside another:

Algorithm AllDifferent($a[0..n-1]$)

$i \leftarrow 1$

$\text{diff} \leftarrow \text{true}$

invariant: $0 \leq i \leq n$ and $\text{diff} \equiv a[0] \dots a[i-1]$ all different

WHILE $i < n$

$j \leftarrow 0$

invariant: $0 \leq i \leq n$ and $\text{diff} \equiv a[0] \dots a[i-1]$ all different AND $a[i]$ different from $a[0] \dots a[j-1]$

WHILE $j < i$

IF $a[j] = a[i]$

$\text{diff} \leftarrow \text{false}$

$j \leftarrow j+1$

$i \leftarrow i+1$

RETURN diff

Here i is the control variable for the outer loop, and j is the control variable for the inner one. Keeping these separate makes it easier to work out when the loops will terminate.

Good practice, for clear code:

- Do not modify a loop control variable in an inner loop.
- Modify the loop control variable in as few places as possible.

The following form is very common:

$i \leftarrow A$

```
WHILE  $i < B$   
  other stuff  
   $i \leftarrow i+1$ 
```

Many languages support this special case directly, e.g. in languages of the C family (including C++, Java and C#), this is written `for (i = A; i < B; i++) ...`

We shall stick with the more general and flexible WHILE form.

Analysing algorithms

We want to measure the number of steps taken by the algorithm (usually in the worst case) as a function of the input size. So we need to consider

- how to measure the size of the input to the algorithm.
- what a fundamental step for the algorithm is.
- what input gives the most number of steps (the worst case).

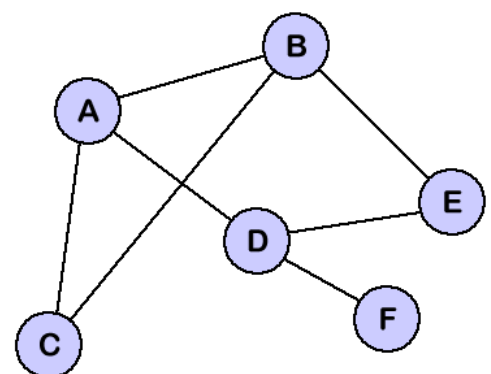
Then we will want to compare the rates of growth of different functions.

Input size

The input size is usually straightforward to decide upon, by simply working out what aspect of the input is used in the algorithm. For example,

- an array of n elements has size n (with an algorithm where the fundamental steps involve elements of the list).
- the size of a graph might be the number of nodes or edges, or both.
- the size of a logical formula might be the number of variables, or the number of connectives.

Some work uses the number of bits in the input.



A graph in the sense of graph theory

Counting steps

To work out the number of steps an algorithm may take (given an input of size n):

- identify the fundamental step the algorithm;
- set up a mathematical formula describing the number of times this step occurs – this should depend on (if anything) the size of the input;
- manipulate this to establish a succinct description of the number of steps.

Comparing cost functions

If we have reduced the costs of two algorithms to cost functions $f(n)$ and $g(n)$, we wish to compare these. Smaller cost is better, but we are most concerned with the costs for large input size n .

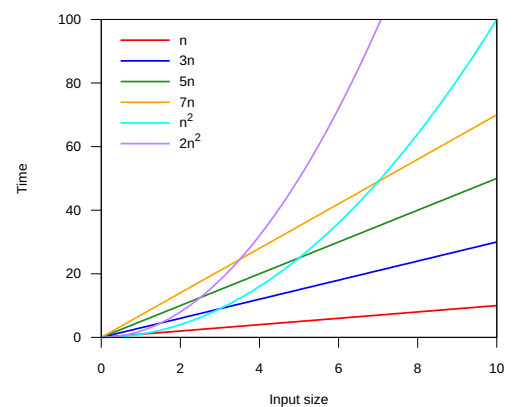
In the graph at right, note that n^2 eventually overtakes $c \cdot n$ for any constant c . We say that n^2 *dominates* $c \cdot n$. Similarly, n^3 dominates $c \cdot n^2$, and so on.

Order-of-growth notation

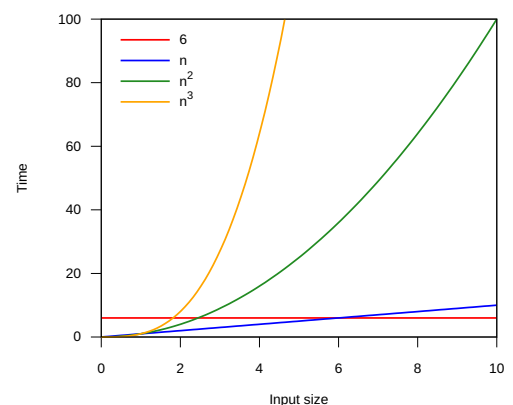
We take a simplified view of a cost function, restricting our attention to the term that grows fastest for large inputs, and ignoring constant factors. Thus we might say that the worst-case complexity of some algorithm is $O(n^2)$, meaning that for large inputs its cost function $f(n)$ grows no faster than n^2 . How large? as large as you like.

We'll cover the formal definition of big-O notation in week 4, but for polynomial functions the complexity class is quite easy to work out:

- 1, 23 and 1,000,000 are all $O(1)$
- $5n + 4$ is $O(n)$
- $20n^2 + 56n + 478$ is $O(n^2)$



$n^2 = n \cdot n$ eventually overtakes $c \cdot n$ for any constant c .



Graphs of some polynomial functions. The asymptotic rate of growth increases with the power k .

- $5n^3 - 34n^2 + 7n + 1002$ is $O(n^3)$
- and so on

In general, a polynomial function (like these) belongs to $O(n^k)$ where k is the largest power of n that occurs. (In week 4 we'll consider other functions.)

The simplified big-O form has a number of properties that make it easy to calculate with, as we'll see in later weeks. For now, we'll consider some polynomial examples.

Example: select a random element

This (rather simple) algorithm selects a random element from an array of length n :

Algorithm RandomElement($a[0..n-1]$)

$i \leftarrow$ random number between 0 and $n-1$
 RETURN $a[i]$

- This algorithm has input of size n , the length of the array.
- The basic step is an array lookup.
- The number of steps taken by this algorithm is always 1.

Thus the complexity of this algorithm is $O(1)$, i.e. *constant* time.

Example: analysis of sequential search

Consider the sequential search algorithm:

Consider a sequential search on this array of 5 elements:

	0	1	2	3	4
a	27	14	77	85	34

- Searching for 27 (the best case), requires one comparison.
- Searching for 14 requires 2 comparisons.
- Searching for 77 requires 3

Algorithm SequentialSearch($a[0..n-1]$, x)

$i \leftarrow 0$

invariant: $0 \leq i \leq n$ and x is not in $a[0] \dots a[i-1]$

WHILE $i < n$ AND $a[i] \neq x$

$i \leftarrow i+1$

RETURN i

comparisons.

- Searching for 85 requires 4 comparisons.
- Searching for 34 requires 5 comparisons.
- Searching for anything not in the array (the worst case) requires 5 comparisons.

For the input size, we'll use the size of the array, n . The steps we'll count are the comparisons of the array elements with x . (We could also count array accesses; in this case it would give exactly the same answer.) Now the number of steps executed by the algorithm varies for different inputs of the same length:

- The worst case occurs when x is not in the array. The algorithm compares it with all the elements, i.e. n steps.
- The best case occurs when x occurs as the first element of the array, so only one comparison is executed.
- If the first x occurs at a random position in the array, the algorithm takes between 1 and n steps, with an average of $(n+1)/2$.

Thus the worst-case complexity of sequential search is $O(n)$, or *linear* time. The average case time when the value is present is also $O(n)$.

One effect of the big-O abstraction is that our choice of input size and step becomes less important. For example, we can count assignments, array accesses or comparisons, and still end up with the same answer.

Example: analysis of AllDifferent

Recall the algorithm for determining whether all the elements of an array are different:

Algorithm AllDifferent($a[0..n-1]$)

$i \leftarrow 1$

diff $\leftarrow$ true

invariant: $0 \leq i \leq n$ and **diff** $\equiv$ $a[0] \dots a[i-1]$ all different

WHILE $i < n$

```

j ← 0
invariant:  $0 \leq i \leq n$  and  $\text{diff} \equiv a[0] \dots a[i-1]$  all different AND  $a[i]$  different from  $a[0] \dots a[j-1]$ 
WHILE j < i
    IF a[j] = a[i]
        diff ← false
    j ← j+1
i ← i+1
RETURN diff

```

Again we'll use the size n of the array as the input size, and count comparisons. This algorithm compares each of the n elements with the $n-1$ other elements, but it only compares each pair once, so it requires $n(n-1)/2$ comparisons, that is $n^2/2 - n/2$. This function is $O(n^2)$, or *quadratic* time.

Complexity classes

We classify algorithms according to the order of their cost function:

$O(1)$ Constant

The algorithm always takes the same amount of time (which is very good). Occurs only in cases of rare good fortune, e.g. first element.

$O(n)$ Linear

Time taken grows in direct proportion to input size (good/OK). Typically for problems requiring a limited number of passes over the input, e.g. sequential search. Any algorithm that examines all its input must take at least linear time.

$O(n^2)$ Quadratic

The time taken grows in proportion to the square of the input size (OK/bad). This often occurs with algorithms involving two nested loops, e.g. AllDifferent, or the sorting methods we'll be discussing next week.

$O(n^3)$ Cubic

The time taken grows in proportion to the cube of the input size (not so good). This often occurs with algorithms involving three nested loops, e.g. naive multiplication of n -

by- n matrices.

$O(n^k)$ Polynomial (general case of the above)

This is the limit of what is usually considered "feasible", though it can be very expensive for larger k . Some algorithms and problems are even worse.

We'll consider other classes in later weeks.
